

6. Ride-Hailing (Uber / Lyft)

Interviewer: Let's design Uber. A rider taps "request ride," we find them a nearby driver, show the car moving on the map, and charge them at the end. Millions of drivers are on the road at any moment, each reporting GPS position every few seconds. Design the backend.

Candidate: This looks like a matching problem, but what actually breaks at scale is upstream of matching: **every driver on Earth streams location into us constantly, whether or not anyone needs them right now.** Let me clarify before committing to a design.

1. Clarifying the requirements

Candidate: How often does a driver ping — every second, every few seconds? Is "nearest available driver" strictly closest, or closest-plus-scoring (rating, vehicle type)? Can a driver ever be offered to two riders at once, even briefly? Do trips ever cross city/region boundaries?

Interviewer: Pings every 4 seconds while on shift. Closest-available is the baseline. **A driver must never be double-booked — that's a hard invariant.** And no, a trip's rider, driver, and route always live within one city.

Candidate: That last answer is a gift — this problem is naturally shardable by city, unlike something that must be globally addressable. Requirements:

Functional: driver streams live location continuously; rider requests pickup+dropoff and gets matched to a nearby available driver; rider sees the driver's position live; fare estimate up front, final fare + surge on completion; trip lifecycle REQUESTED → MATCHED → ONGOING → COMPLETED/CANCELLED .

Non-functional: location pings dominate write volume by orders of magnitude over trip writes — this is a "survive the writes" problem, not a "scale the reads" one. Match within 1-2s. Never double-book a driver (strong consistency on assignment specifically). Location a few seconds stale is fine (eventual). City-local by nature → cheap regional sharding.

2. Estimation — the number that defines the design

```
5M active drivers, pinging every 4s while online:
  location writes/sec ~= 5,000,000 / 4 ~= 1.25M writes/sec  <-- the firehose

10M ride requests/day:
  avg ~= 10M/86,400 ~= 115/sec;  peak (5-10x) ~= ~1,000/sec

Trip record ~1KB -> 10GB/day, trivial for a relational DB.
Location record ~50 bytes, but we only care about the LATEST point --
no accumulating history in the hot path, just overwrite in place.

=> 1.25M/s of ephemeral overwrite traffic vs ~1,000/s of durable
    transactional traffic. Two access patterns 1,000x apart -- they
    cannot share a store.
```

3. V1 — the naive design, and where it dies

```
[Driver app] --POST /location, every 4s--> [App server] --> UPDATE drivers
                                                    SET lat=?,lng=? (Postgres)

[Rider app] --POST /rides--> [App server] -->
  SELECT * FROM drivers WHERE status='AVAILABLE'
  ORDER BY (lat-rlat)^2+(lng-rlng)^2 ASC LIMIT 1;  -- scans every driver!
```

Why it dies, two ways: (1) 1.25M row UPDATE s/sec pay WAL/MVCC cost for data we don't need to persist durably — no relational DB sustains that without an absurd fleet. (2) "closest driver" as distance-to-every-row is an O(N) scan per request; a B-tree on (lat, lng) doesn't prune a 2D radius search. Root cause: **treating spatial, ephemeral, high-churn state as relational data.**

4. The fix — split the store by access pattern

1. Move live location into an in-memory store with a **spatial index** — fast overwrite, no durability requirement.
2. Keep the relational DB only for what needs ACID: trips, payments, driver status.

```
Driver app --WS ping--> Location Service --GEOADD--> Redis (geo-index, per city)
                                                    | GEOSEARCH
                                                    ▼
Rider app --request--> Dispatch/Matching Service --candidates-->
                                                    | conditional UPDATE (atomic claim)
                                                    ▼
                                                    Postgres (city shard): drivers, trips, payments
```

Redis answers "which drivers are near here right now" fast; Postgres answers "is this specific driver actually free, and can I lock him to this trip" correctly. Neither store does the other's job.

5. Geo-indexing — finding nearby drivers without a scan

Interviewer: Concretely — how do you find drivers within 2km without scanning the city?

Candidate: Three real options.

- **Geohash:** encode (lat, lng) into a base-32 string where shared prefixes mean proximity; radius search = "my cell + ~8 neighbors." This is exactly what Redis's `GEOADD / GESEARCH` uses internally.
- **QuadTree:** recursively subdivide space so dense areas get finer cells automatically. More adaptive, but it's a custom tree you build and rebalance yourself.
- **Uber H3 (hex grid):** every neighbor cell is equidistant from center — squares distort "nearest" near diagonal boundaries. Uber built H3 specifically so surge zones and dispatch get consistent neighbor distances.

Pick: Redis geohash (`GEOADD / GESEARCH`) as the day-one implementation — one atomic, off-the-shelf system solves the write-firehose problem *and* the spatial query in one place, no index to keep in sync. I'd reach for H3 only when custom hex business-zones (surge boundaries) matter. QuadTree is rejected here — we'd be hand-building a spatial structure Redis already gives us atomically.

```
GEOADD drivers:city:sfo lng lat driver_id      <- every ~4s per driver
GESEARCH drivers:city:sfo FROMLONLAT lng lat BYRADIUS 2 km ASC COUNT 10
```

cell A	cell B	cell C
cell D	RIDER	cell F
cell G	cell H	cell I

Each cell ~ a geohash prefix bucket.
Query searches rider's cell + neighbors,
ranks by real distance, returns top-K —
never touches cell J across town.

One sorted-set key PER CITY keeps each structure small (thousands, not millions, of points) and bounds the search radius instead of scanning a global index.

Cell-size tradeoff: too coarse → thousands of candidates to rank; too fine → sparse suburbs need to union many near-empty neighbor cells. Tune precision per city density.

6. Matching — and the double-booking race

Interviewer: Two riders near the same driver request at once, and `GESEARCH` returns the same driver to both. What happens?

Candidate: The geo-index answers "who's *probably* free and *near*" — not "who's free right now." Only the second question gets to be a lock.

```

BAD -- read-then-write race:
T1: GEOSEARCH -> D77 (looks AVAILABLE)
T2: GEOSEARCH -> D77 (same instant, also looks AVAILABLE)
T1: UPDATE drivers SET status='ON_TRIP' WHERE id=D77;   commits trip A
T2: UPDATE drivers SET status='ON_TRIP' WHERE id=D77;   commits trip B <- double-booked!

```

Fix — the geo-index only produces candidates; the claim is one atomic conditional write:

```

UPDATE drivers SET status='ON_TRIP', current_trip_id=:tripId
WHERE driver_id=:candidateId AND status='AVAILABLE';
-- rows_affected = 1 -> you won; create the trip
-- rows_affected = 0 -> lost the race; try next candidate

```

```

Dispatch(A) --GEOSEARCH--> [D77, D42, D91]
      | CAS on D77 WHERE AVAILABLE
      ▼
rows_affected=1 -- WINS
create trip A, notify driver

```

```

Dispatch(B) --GEOSEARCH--> [D77, D65, D91]
      | CAS on D77 WHERE AVAILABLE
      ▼
rows_affected=0 -- LOSES
falls to D65, retries CAS

```

No distributed lock manager needed — the critical section is one row, and the DB's own row-locking serializes the writers; the loser just gets `0` rows, a normal signal to advance to the next candidate. Same family of trick as an atomic `DECR` on a scarce counter: push the decision into one atomic primitive instead of read-then-write in application code. An equally valid variant: do the claim as a Redis `SETNX` /Lua script for memory-speed, then persist the trip in Postgres right after.

7. Scaling the firehose further

- **Shard by city** (`drivers:city:sfo`, `drivers:city:nyc`, ...) — a business-given shard boundary, not hash-and-hope. 1.25M/s splits cleanly across cities.
- **WebSocket, not repeated HTTP**, for pings — one persistent connection per driver-shift avoids handshake overhead at 5M concurrent drivers.
- **Overwrite, not append** — `GEOADD` replaces the previous position; hot-store size is $O(\text{active drivers})$, not $O(\text{pings ever sent})$.
- **No synchronous durability on the write path** — losing a few seconds of positions on crash is fine since drivers resend every 4s.
- **Async fan-out to riders** via pub/sub off the same location stream, not per-rider polling reads against Redis.

8. Multi-region — city is the natural shard

[SFO region]	[NYC region]	[LON region]
Redis geo (SFO)	Redis geo (NYC)	Redis geo (LON)
Postgres shard (SFO)	Postgres shard (NYC)	Postgres shard (LON)
Dispatch service	Dispatch service	Dispatch service

Rider's app connects to home city's cluster (or nearest region while traveling). Trip creation, matching, payment all happen inside that region -- zero cross-region calls on the hot path.

Cross-region need: rider's global trip-history rollup for the app's "past trips" screen -- replicated async into a central store, off the critical path of any live trip.

A Manhattan rush-hour surge only needs more capacity in the NYC cluster, not a global bump — and a regional outage never touches another city's ability to match rides.

9. Latency — where the milliseconds go

```
Rider taps "request"
-> gateway auth+routing                ~10-20ms
-> GEOSEARCH candidates (in-memory, city-local) ~5-15ms
-> rank candidates (distance+rating+ETA) ~10-30ms
-> conditional UPDATE claim (CAS, DB round trip) ~10-30ms per attempt
-> push offer to driver device over WS ~50-150ms
-> driver accepts / auto-timeout        budgeted separately (~10-15s)
```

Server-side "find and lock a driver" budget: well under 200ms. Most of the rider's perceived wait is the driver's accept window, bounded by a timeout that falls through to the next candidate.

10. Consistency model

- **Strong:** the assignment claim — the conditional UPDATE is the one place we need a real guarantee, exactly one trip ever wins a given driver.
 - **Eventual:** driver location (a few seconds stale is fine — the store was chosen precisely because it doesn't need durability or cross-replica immediacy) and surge multiplier (rolling ratio recomputed every ~30s — a pricing signal, not a lock).
 - **The seam:** matching *reads* an eventually-consistent signal (the geo-index) but *commits* with a strongly-consistent write (the CAS). An eventually-consistent read never gets to be the final word on a hard invariant — it only produces candidates re-checked atomically at the moment of truth.
-

11. Async — what moves off the critical path

```
[ Trip matched: rider+driver+CAS committed ]
├─ push notification to driver device          (fire-and-forget)
├─ emit "trip-matched" -> Kafka (trip-events)
│   └─ analytics / surge-signal aggregator
│       └─ notification service (SMS/email)
│           └─ archival worker -> wide-column store after COMPLETED
└─ respond to rider immediately: "matched, ETA 4 min"
```

- **Rider position fan-out is pub/sub, not per-rider polling** — one write fans out to whoever is watching that driver, without re-querying the geo-index per subscriber per tick.
- **Billing/receipts/archival are a pipeline, not a blocking chain** — rider hears "trip complete" immediately; fare capture, receipt, and cold storage happen behind Kafka.
- **Why Kafka:** ordered, replayable, partitioned log (by `trip_id` /city) so a slow notification provider or analytics job lags/retries without touching the live trip path, and we can reprocess events for reconciliation (did every matched trip get billed?).

12. Caching — and why Redis here isn't "a cache"

Worth being precise: **Redis is not a cache in front of a DB for location** — there's no SQL table of live coordinates it's caching. Redis *is* the system of record for "where is this driver now." That's a deliberate departure from cache-aside.

```
LOCATION (Redis = source of truth, no fallback store):
  Driver --WS ping--> Location Svc --GEOADD--> Redis
  Rider <--WS push-- Location Svc <--pub/sub-- (fanned out from the same write)

TRIP STATUS (normal cache-aside on Postgres):
  read -> check Redis (short TTL) -> hit: return; miss: read Postgres, populate, return
  write -> write Postgres, then invalidate/update the cache key
```

Trip status is read far more than written once underway, so cache-aside with short TTL and write-time invalidation is the standard choice — only location breaks the usual split, because there's no durable backing store it could be stale relative to.

13. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Live driver location	Redis, geo-indexed, sharded per city	1.25M overwrite writes/sec need memory-speed atomic upserts with a built-in spatial index; a few seconds of data loss is fine. <i>Not a relational DB</i> — that write rate demands WAL/durability we don't need, and a B-tree can't prune a 2D radius search.
Geo-index structure	Geohash sorted sets (Redis native); H3 as the upgrade for	Geohash gives atomic insert+radius-search in one system, no index to sync. <i>Not a hand-rolled QuadTree</i> — more

Concern	Choice	Why this, and why not the alternative
	custom hex zones	adaptive, but now you own tree rebalancing yourself. <i>H3</i> wins when surge zones/ops dashboards need uniform-distance neighbor cells.
Trips, payments, profiles	Relational DB (Postgres/MySQL), sharded per city	Assigning a driver + creating a trip must be one atomic, auditable operation; money needs ACID and joins. ~1K writes/sec peak is trivial for SQL. <i>Not the same store as location</i> — mixing the firehose with transactional data would starve it of I/O it doesn't need to share.
Anti-double-booking	Single-row conditional UPDATE ... WHERE status='AVAILABLE' (CAS)	Critical section is one row; DB row-locking serializes claimants for free — no lock manager needed. <i>Not read-then-write in app code</i> — that's exactly the race that double-books a driver.
Location ping transport	WebSocket	Persistent duplex connection avoids per-ping handshake at 5M concurrent drivers and lets the server push to riders. <i>Not repeated HTTP</i> — setup overhead would multiply the firehose problem.
Ride request API	REST/HTTPS	Simple resource semantics, cacheable GETs, easy on mobile. <i>Not WS for everything</i> — request/response calls don't need a persistent connection.
Event backbone	Kafka , partitioned by trip_id /city	Durable, replayable log feeding billing/notifications/analytics; a flaky downstream must never stall matching. <i>Not synchronous calls per service</i> — one slow provider would then block trip completion.
Rider position fan-out	Pub/Sub (Redis or Kafka) keyed by driver_id	One update fans out to N watchers without re-querying the geo-index per subscriber.
Cold trip history	Wide-column store (Cassandra/Bigtable) or S3+Parquet, async	Write-once, scan/lookup access — keeps hot SQL tables small.
Idempotency	Idempotency-Key on POST /rides and payment capture	A retried ride request over a flaky network must not create two trips; a retried payment must not double-charge.

Say-it-out-loud justification: "The design pivots on splitting storage by access pattern: Redis with a geo-index absorbs the 1.25M/s location firehose in memory with atomic, disposable overwrites, while a sharded relational DB holds trips and payments where ACID matters. The geo-index only ever produces candidates — the never-double-book invariant is enforced by one atomic conditional UPDATE, a compare-and-swap on a single row, so two riders racing for the same driver can never both win. Cities are natural shard boundaries, so both stores scale regionally with zero cross-region matching. WebSocket carries continuous streams; REST carries discrete requests; Kafka carries everything that can happen after the rider already knows they're matched."

14. Failure modes & graceful degradation

Failure	What happens	Mitigation
Driver's WS drops mid-shift	Location goes stale	TTL-expire the geo entry after N seconds of silence; drop from GEOSEARCH — never dispatch to a driver who may be gone
Dispatch offer times out / declined	Rider's request stalls	Retry next candidate from the <i>same</i> already-fetched GEOSEARCH result
No drivers found nearby	Rider sees nothing	Progressively widen radius; else clean "no drivers available"
Redis shard for a city down	Matching briefly impaired there	Fast replica failover (tuned for speed, not durability) — a short matching gap beats losing trips/payments durability, which lives elsewhere
Postgres primary (city shard) down	Can't commit assignments there	Leader failover; reads serve from replicas, but the CAS write waits for a primary — no eventual consistency on "is this driver taken"
Payment fails after trip ends	Rider shouldn't be stuck	Mark COMPLETED regardless; retry payment async + idempotently; never block exit over a billing retry
Whole region/city cluster down	Only that city's rides affected	Matching was always city-local — no blast radius elsewhere

Design principle: a stale or missing *location* fails safe — worst case a slower match. A failure must never fail unsafe on *assignment* or *payment* — that's where the strong- consistency budget was spent.

15. Follow-up curveballs

Interviewer: A driver's GPS jumps 500m in one ping — clearly noise. How do you handle it?

Candidate: Don't trust a single ping blindly — smooth with a Kalman filter, or a cheap heuristic ("reject a jump implying >150km/h since the last ping"), before writing to the geo-index. Never let a GPS glitch send a rider to a driver's phantom location.

Interviewer: Is ETA the geo-index's job?

Candidate: No — geo-index answers straight-line/grid proximity, not driving time. A separate **routing engine** (road graph + live traffic) re-ranks the top-K geo-candidates by real drive time. Geo-index narrows millions to ~10 fast; routing does the expensive precise ranking on that small set.

Interviewer: Add carpooling — one driver, multiple riders, different destinations.

Candidate: Trip model becomes one-driver-to-many-riders; matching becomes a mini bin-packing/routing problem — "which in-progress route can absorb this rider within an acceptable detour

for everyone aboard." The atomic-claim principle is unchanged (still need a race-free reservation of a remaining seat), but candidate generation now also reasons about existing passengers' routes, not just proximity.

Interviewer: How do you compute surge without re-locking every request?

Candidate: Background aggregate, not per-request computation: a job tracks (open requests ÷ available drivers) per city cell every ~30s and writes a multiplier per cell to a fast-read key. A ride request just reads the current multiplier — eventually-consistent, no locking, because pricing doesn't carry the same never-double invariant as assignment.

16. Deep-dive: "Why not just store live locations in Postgres/PostGIS?"

Interviewer: PostGIS is a real, mature geospatial engine — GiST indexes, `ST_DWithin`, KNN operators, the works. Why not just add a `location` column with a spatial index and query it directly? Then you'd have ACID and one fewer system to operate.

Candidate: PostGIS is genuinely the right tool for geospatial queries **at rest** — I'd reach for it for "which delivery zones does this polygon overlap" or "drivers who completed a trip in this geofence last month." The problem here isn't the query engine's *correctness*, it's that our workload is dominated by the **write side**, and that's where a relational spatial index falls over.

The firehose meets index maintenance

Every location ping is a **write to an indexed column**, and spatial indexes are not cheap to update in place:

1.25M location writes/sec, EACH one touches a GiST-indexed row:

```
UPDATE drivers SET geom = ST_Point(:lng,:lat) WHERE driver_id = :id;
```

Under MVCC, Postgres doesn't overwrite the row in place:

1. old tuple marked dead, NEW tuple inserted (heap bloat)
2. GiST index entry for the OLD position must be deleted, a NEW entry inserted at the new position (no cheap in-place move for a tree-structured spatial index)
3. WAL record written for the heap change AND the index change
4. autovacuum must later reclaim the dead tuples --
at 1.25M/s, vacuum can never catch up; heap and index both bloat without bound, and every one of those also has to replicate

Net: ONE logical "driver moved 10 meters" turns into a WAL write, an index delete+insert, and a future vacuum pass -- write AMPLIFICATION, not a single cheap overwrite.

Compare that to Redis: `GEOADD` mutates one entry in an in-memory sorted set (a geohash score), no WAL, no MVCC, no vacuum, no replication unless you choose it. It's designed to be overwritten constantly because it holds *only the latest position* — there's no history to preserve, so there's nothing to amplify.

To be precise about what's separate

This is exactly why the **trips database stays relational** (§13) — trip creation is ~1,000 writes/sec, each one rare relative to a trip's multi-minute lifetime, and needs joins, ACID, and durability because money and legal records are on the line. Location is the opposite shape: 1,000x the write rate, each write superseding the last, needing none of those guarantees. Same database engine, wildly different access pattern — that mismatch is the whole argument, not "Postgres is slow."

Concern	Relational / PostGIS	Redis geo / in-memory grid
Write model	UPDATE + index maintenance (GiST delete+insert), MVCC creates new tuple versions	In-place overwrite of one sorted-set entry; no versions to keep
Write amplification at 1.25M/s	High — WAL + index churn + eventual vacuum, all scaling with write rate	Minimal — one memory write per ping, nothing to reclaim
Durability	Full ACID, survives crash via WAL replay — but we don't need this for a value that's stale again in 4s	None by default — acceptable, since a lost point is refreshed on the next ping
Throughput ceiling per node	Bounded by disk I/O + vacuum throughput; spatial index writes are the expensive kind of write	Bounded by memory bandwidth/CPU; scales by adding shards, not by fighting vacuum
Query pattern fit	Excellent for complex, ad-hoc, or historical spatial queries (polygons, joins, "as of last month")	Excellent for one narrow, extremely hot query: "who's near this point right now"
Operational cost under this load	Constant vacuum tuning, index bloat monitoring, replication lag from WAL volume	Sharded, disposable, restart just means "wait 4s for pings to refill it"
Right fit here?	Yes — for the trips/payments DB, which is low-volume and durable	Yes — for live location, which is high-volume and disposable

Rule of thumb: *Relational + spatial index when writes are infrequent and each one matters (durable, joinable, historical). In-memory geo-index when writes are a firehose of disposable, constantly-superseded state. The two aren't competing for the same job in this system — one holds the ledger, the other holds "right now."*

17. Deep-dive: the matching race — one driver, two offers at once

Interviewer: Walk me through this in real detail. Two riders both have the same nearby driver as their top candidate at almost the same instant. Show me exactly how the race happens, how your fix actually closes it, what happens if the driver declines or just doesn't answer, and how you stop one popular driver from getting buried under pending offers from every rider nearby.

Candidate: Let me split this into four parts: the race itself, the atomic fix, the decline/timeout lifecycle, and the "hot driver" starvation problem — because a correct CAS alone doesn't solve all four.

Part 1 — how the naive "find nearest, assign" races

time	Dispatch(Rider A)	Dispatch(Rider B)
t0	GEOSEARCH -> [D77, D42, D91]	
t0	(same instant, different process)	GEOSEARCH -> [D77, D65]
t1	read driver D77: status = AVAILABLE	
t1		read driver D77: stat
t2	app decides "assign D77 to A"	
t2		app decides "assign D
t3	UPDATE drivers SET status='ON_TRIP', current_trip_id=tripA WHERE id=D77;	(no WHERE guard on status)
t3		UPDATE drivers SET st current_trip_id=trip
t4	notify driver: "trip A incoming"	
t4		notify driver: "trip
	=> D77 now has current_trip_id = tripB (last write won), but the app already told the DRIVER'S CLIENT and RIDER A that trip A is confirmed. Two riders both believe they have D77.	

The bug isn't a missing lock in the database sense — it's that the code **read a status, decided in application memory, then wrote**, with no guarantee nothing changed between the read and the write. Two processes can both pass the read and both "win" in their own head.

Part 2 — the atomic-assignment fix

Collapse read-decide-write into one statement so the database's own row lock is the serialization point, not application logic:

```
UPDATE drivers
  SET status = 'OFFERED', current_trip_id = :tripId, offer_expires_at = now() + interval '15s'
  WHERE driver_id = :candidateId AND status = 'AVAILABLE';
-- rows_affected = 1 -> you hold the offer; push it to the driver's device
-- rows_affected = 0 -> someone else already moved this driver; try next candidate
```

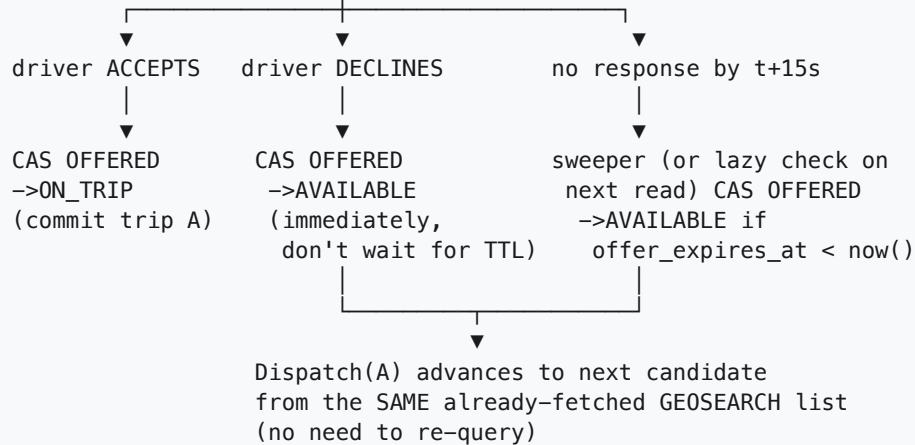
t0	Dispatch(A): CAS AVAILABLE->OFFERED on D77	rows_affected=1	WINS
t0	Dispatch(B): CAS AVAILABLE->OFFERED on D77	rows_affected=0	LOSES (row already OFFERED -> falls to D65, retries CAS there)

Note the state machine now has an explicit **OFFERED** state between **AVAILABLE** and **ON_TRIP** — the CAS only grants a *hold*, not a committed trip. That distinction is what makes decline/timeout handling clean.

Part 3 — driver declines or times out

An offer can't stay pending forever, and it can't require the rider to sit there not knowing:

D77: AVAILABLE --CAS(A)--> OFFERED(tripA, expires t+15s)



Every transition is the same primitive: a conditional `UPDATE ... WHERE status='OFFERED' AND current_trip_id=:tripId`, so a late/duplicate decline or a race between "driver taps accept" and "TTL sweeper fires" can't double-apply — only the first one to match the `WHERE` wins, the second gets `rows_affected=0` and is a no-op.

Part 4 — don't let one driver get buried under pending offers

The CAS prevents *double-booking*, but it doesn't by itself prevent a subtler failure: in a dense area with more riders than drivers, the **same driver is the #1 candidate for many riders' lists**. Even with correct CAS semantics, that means every rider-dispatch except one spends a round trip failing against D77 before trying candidate #2 — D77 becomes a hot key that many requests contend on, and worse, whichever rider wins gets D77 while everyone else's *effective* candidate list quietly degrades to their #2, #3 choice, request after request, in the same busy block.

Ranked fixes:

1. **Do nothing, just retry next candidate on `rows_affected=0`**. Correct, no double-booking — but under sustained high contention (rush hour, a stadium letting out) the same driver ID absorbs a disproportionate number of failed CAS attempts, and riders behind the winner all pay an extra round trip before reaching a driver who was actually free.
2. **Per-driver distributed lock (e.g. Redis `SET NX` with a TTL) held for the duration of the offer.** Explicit and easy to reason about, but it's one more moving part than the CAS already gives us for free via the DB's row lock, and a lock needs its own TTL/expiry discipline to avoid a crashed dispatcher stranding a driver as permanently "locked."
3. **Randomize among the top-K near-equidistant candidates instead of always targeting the single nearest driver** (the same idea as power-of-two-choices from the burger problem, §12/§13 there). Rather than every nearby rider's dispatch converging on literally the same driver ID first, spread first-attempts across, say, the top 3–5 candidates within a small distance band. This doesn't change the CAS logic at all — it just reduces how often many concurrent dispatches pick the *same* row as their first attempt, cutting contention at the source instead of absorbing it after the fact. ✅ **recommended, layered on top of #1**
4. **Cap concurrent offers a driver can be "top candidate" for by pre-filtering candidates whose ETA-to-pickup is already too long given queueing** — more complex, solves a slightly different problem (fairness of who gets offered first), and I wouldn't reach for it before #3.

The atomic CAS (Part 2) is non-negotiable — it's what makes the invariant "never double-book" structurally true. The hot-driver problem is a performance/fairness concern layered on top, and lightweight randomization among near-equal candidates is the cheapest fix that doesn't add a new subsystem.

What to say out loud: *"Double-booking is closed by collapsing read-decide-write into one conditional UPDATE — the database's own row lock is the only serialization we need, so the loser of a race gets a clean zero-rows-affected signal instead of corrupting state. Decline and timeout are the same primitive in reverse: a conditional UPDATE back to AVAILABLE, guarded so a late decline and a TTL sweep can't both apply. The remaining risk isn't correctness, it's contention — one popular driver being everyone's first CAS attempt in a dense area — and I'd fix that by spreading first-attempts across a small band of near-equidistant candidates rather than always targeting the single nearest driver, the same power-of-few-choices trick that keeps any single hot key from absorbing all the contention."*

Summary — the mental model

Ride-hailing is two problems wearing one trench coat: **a write firehose of ephemeral location data**, and **a small, strict, transactional matching/payment problem** — the design is about never letting either contaminate the other. Location lives in an in-memory, geo-indexed store (Redis geohash, H3 for custom zones) sharded per city, because it's disposable and needs spatial radius queries, not ACID. Trips and payments live in a relational store sharded per city, because they need ACID and don't face millions-per-second write pressure. The geo-index is never the final word on availability — it proposes candidates; the "this driver is now taken" decision is one atomic conditional UPDATE that makes double-booking structurally impossible without any distributed lock manager. Cities are shard boundaries the business gives you for free. Everything after "you're matched" — billing, receipts, archival, analytics — moves async behind Kafka so none of it slows the one interaction that must be fast: finding and locking a driver.